The method IO#each_line calls the block with each line from the file. In the next example, we'll make the original newlines visible using String#dump, which returns the string in double-quotes with escape characters, so you can see that we're not cheating:

```
File.open("testfile") do |file|
  file.each_line { |line| puts "Got #{line.dump}" }
end
```

produces:

```
Got "This is line one\n"
Got "This is line two\n"
Got "This is line three\n"
Got "And so on...\n"
```

The each_line method includes the line ending at the end of each line of data. That's why you see the \n characters in the output of the previous example. You don't have to use \n as the separator, though. You can pass each_line an argument, any sequence of characters. The method will use that argument as a line separator, and will break up the input accordingly, returning the separator at the end of each line of data. In the next example, we'll use the character e as the line separator:

```
File.open("testfile") do |file|
  file.each_line("e") { |line|  puts "Got #{ line.dump }" }
end
```

produces:

```
Got "This is line"
Got " one"
Got "\nThis is line"
Got " two\nThis is line"
Got " thre"
Got "e"
Got "\nAnd so on...\n"
```

If you combine the idea of an iterator with the autoclosing block feature, you get IO.foreach, or the subclass method File.foreach. This method takes the name of an I/O source, opens it for reading, calls the iterator once for every line in the file, and then closes the file automatically:

```
File.foreach("testfile") { |line| puts line }
```

produces:

```
This is line one
This is line two
This is line three
And so on...
```

Or, if you prefer, you can retrieve an entire file into a string:

```
str = IO.read("testfile")
str.length # => 66
str[0, 30] # => "This is line one\nThis is line "
```

Or into an array of lines:

```
arr = IO.readlines("testfile")
arr.length # => 4
arr[0]     # => "This is line one\n"
```